

The FDE Toolkit

An Orchestrator-Centred Design

A design for an AI-native workbench in which a central Orchestrator coordinates a team of specialist agents to move an engagement through the ten-stage FDE lifecycle — continuously converting conversation into governed, structured artefacts.

Design specification · v1.0 - Ajit Jaokar 22 July 2026

By [Ajit Jaokar](#) Erdos Research

Contents

Contents	2
Background	3
Why an Orchestrator, not a workflow engine	3
Layered architecture	5
4 The Orchestrator in detail	6
5 The agent roster	7
6 The information model	8
7 The conversation–artefact loop	9
8 Orchestration patterns	10
9 Governance as a cross-cutting concern	10
10 Build sequence	13

Background

This document creates a design of an Open Forward Deployed Engineer (FDE) toolkit.

As envisaged here, the FDE Toolkit is not a set of separate tools, and it is not a set of forms (CRUD). Instead, it is a single AI-native workbench in which a central Orchestrator coordinates a team of specialist agents to move an engagement through the multiple stages (in this case ten stages). These stages are : Discover, Purpose, Knowledge, Reasoning, Action, Agent Design, Governance, Build, Evaluate, Evolve.

The organising idea is the **Orchestrator**. The Orchestrator does not perform the specialist work itself. It decides *what* needs doing, *by whom*, in *what order*, with *what context*, under *what controls* — and it keeps the whole engagement coherent as it evolves.

In this sense, the toolkit is ideally suited for developers but also non developers (domain experts). The implementation is also technology agnostic.

The platform is built as an open system primarily for our students

The central artefact

The central artefact of the system is not a record in a database. It is a conversation that continuously generates and refines structured artefacts. Users should feel that they are collaborating with a team of specialist assistants that progressively transform an ambiguous business problem into a deployable, governed AI solution — not filling in forms.

This document develops the Orchestrator concept into a complete design: the layered architecture, the Orchestrator's responsibilities, the agent roster, the shared information model, the conversation–artefact loop, the orchestration patterns that sequence the work, the treatment of governance as a cross-cutting concern, and the build sequence.

Why an Orchestrator, not a workflow engine

A fixed workflow engine or a menu of independent tools would be the obvious way to implement ten lifecycle stages. Both are not the ideal choice for this problem, for four reasons.

- **The lifecycle is non-linear.** FDEs loop back constantly: an evaluation reveals a knowledge gap, so the engagement returns to Knowledge; a governance finding invalidates a proposed workflow. A hard-wired sequence cannot express this. An Orchestrator reasons about the current state and routes dynamically.
- **Context must be shared.** The Governance agent needs what Discovery found; the Evaluation agent needs the Reasoning models. The Orchestrator is the single custodian of shared context, assembling the right slice for each agent call.
- **Artefacts are interdependent.** The ontology feeds the workflow graph, which feeds the agent design. The Orchestrator maintains referential integrity across artefacts through the knowledge graph, and propagates change when an upstream artefact moves.
- **Governance must be enforced between stages**, not bolted on afterwards. The Orchestrator is the natural gatekeeper: it holds the stage gates, applies guardrails, and requires human sign-off where policy demands it.

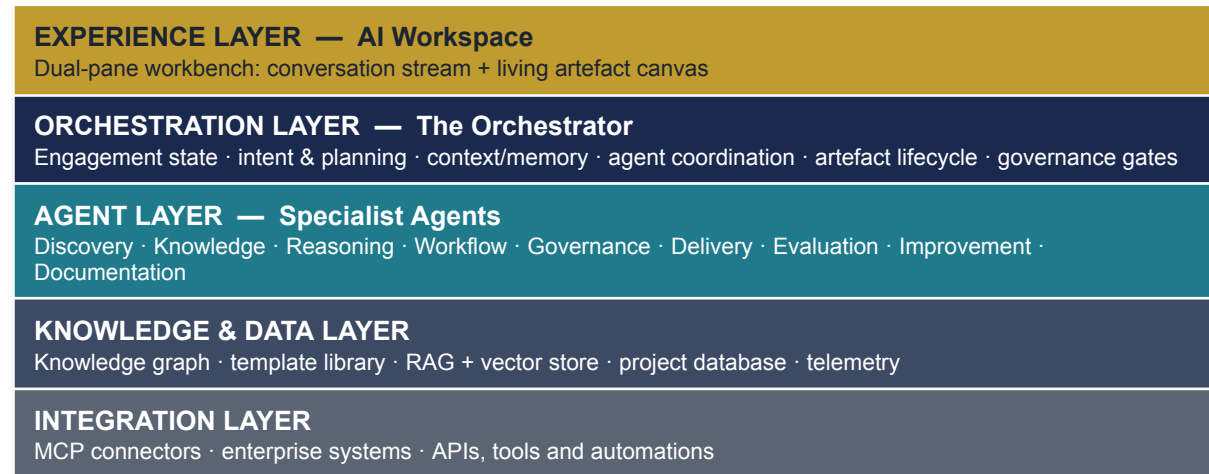
Design principle

Spend 60–70% of the design effort before writing a single line of code. If the methodology, information model, artefacts and user journeys are well designed, the choice of frontend

framework, backend stack or database becomes a straightforward engineering decision rather than the core design challenge.

Layered architecture

The system resolves into five layers. The Orchestrator sits at the centre — everything the user does passes through it, and everything the agents produce is committed through it.



Reading the stack

- **Experience layer.** A dual-pane workbench. On one side a conversation stream; on the other a living canvas of artefacts — workflow maps, decision models, governance controls, architectures — that update as the conversation proceeds. The FDE uploads transcripts and documents; the system extracts, asks follow-up questions, and the FDE edits.
- **Orchestration layer.** The Orchestrator and its working memory and engagement-state store. This is the subject of Section 4.
- **Agent layer.** Specialist agents, each owning one artefact domain. Invoked by the Orchestrator, never directly by the user.
- **Knowledge & data layer.** The knowledge graph (the instantiated information model), the template library, RAG and vector store, the project database, and telemetry.
- **Integration layer.** MCP connectors and APIs to enterprise systems. The Action, Build and Evaluate stages reach through here to real tools and data.

4 The Orchestrator in detail

The Orchestrator decomposes into six responsibilities. Together they let it run an engagement end to end while keeping it coherent and governed.

Responsibility	What it does
Engagement state	Knows where the engagement sits across the ten stages. Each stage has entry and exit criteria — the stage “Outputs” become completion gates. The Orchestrator advances, pauses, or reopens stages as evidence accumulates.
Intent & planning	Interprets the user’s conversational input, decides which agent or agents to invoke, and plans multi-step sequences (e.g. “extract an ontology, then draft candidate workflows, then have Governance review them”).
Context & memory	Assembles the right slice of the knowledge graph plus conversation history as context for each agent call — the core act of context engineering — and persists what comes back.
Agent coordination	Invokes specialist agents using multi-agent patterns: sequential hand-off, parallel fan-out, hierarchical delegation, and generator–critic loops (Section 8).
Artefact lifecycle	Runs the conversation → artefact → graph → reasoning → updated-artefact loop. Every artefact is a first-class, versioned object with provenance.
Governance & guardrails	Applies policies, access controls, human-in-the-loop approvals and stage gates; maintains the audit trail. Governance is enforced here, not left to individual agents.

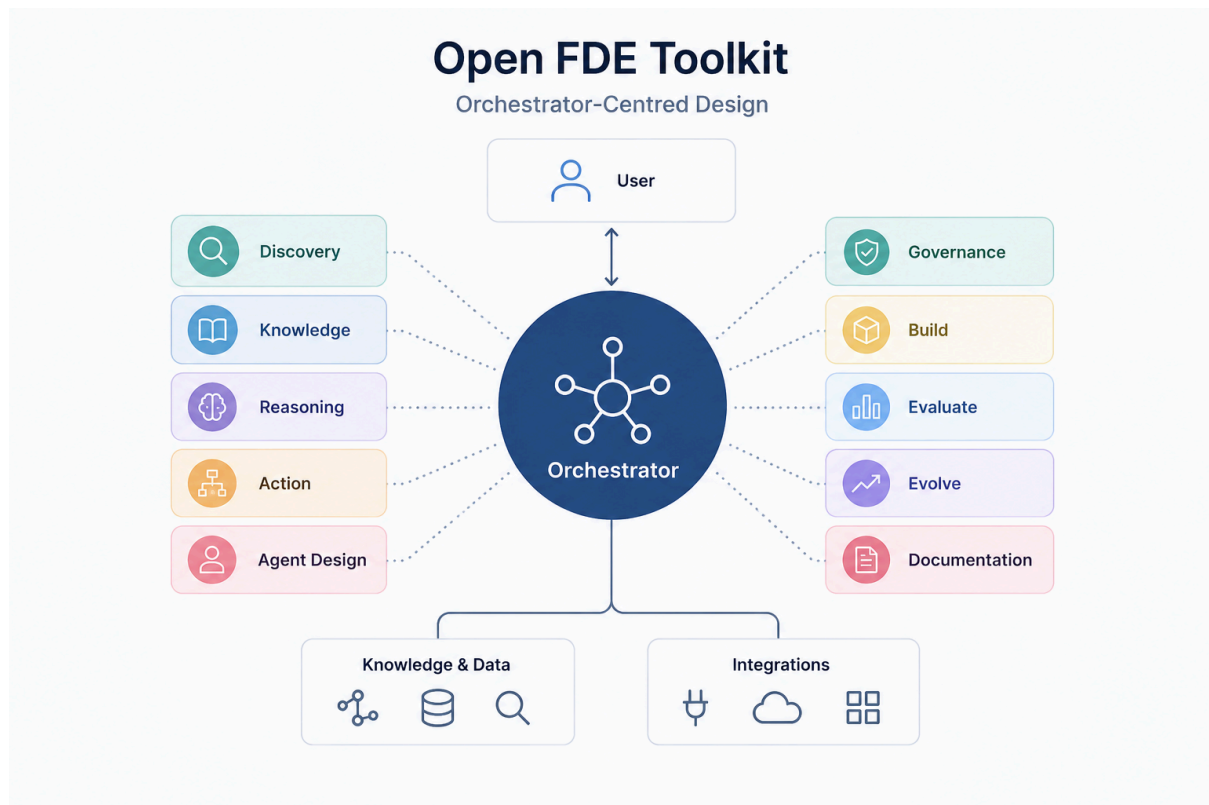
The Orchestrator is not the Agent-Design Agent

A subtle but important distinction: the Agent-Design Agent (Stage 6) designs the agents that will run in the client’s deployed solution. The Orchestrator runs the toolkit itself. One is an artefact the toolkit produces; the other is the machinery that produces it.

5 The agent roster

Each agent owns one artefact domain and maps to the lifecycle. The Orchestrator calls them; they never call the user. Two agents — Governance and Documentation — are cross-cutting: the Orchestrator can invoke them at any stage.

Agent	Lifecycle stage(s)	Owns
Discovery	Discover · Purpose	Stakeholders, objectives, success criteria, value hypothesis, scope
Knowledge	Knowledge	Knowledge inventory, ontology, data map, RAG sources
Reasoning	Reasoning	Decision models, business rules, decision boundaries
Workflow	Action	Workflow graph, tools, automations, integration inventory
Agent-Design	Agent Design	Agent architecture, human–AI operating model, orchestration and memory patterns
Governance	Governance (cross-cutting)	Policy, guardrails, access matrix, risk register, approval workflows
Delivery	Build	PRD, architecture, backlog, specifications, tests
Evaluation	Evaluate	Evaluation framework, evals library, benchmarks, telemetry dashboards
Improvement	Evolve	Experiments, A/B tests, optimisation backlog, feedback loops
Documentation	All (cross-cutting)	Renders current graph state into deliverables on demand



6 The information model

The information model is the schema of the knowledge graph — the shared substrate the Orchestrator reasons over and every agent reads from and writes to. It is built by asking two questions: “*what nouns exist?*” and “*what relates to what?*”

Core entities

Project · Organisation · Stakeholder · Interview · Requirement · Knowledge Source · Ontology · Dataset · Decision · Policy · Risk · Workflow · Task · Agent · Tool · API · Evaluation · Metric · Experiment · Workflow Version · Evidence · Deliverable · Pattern.

Key relationships

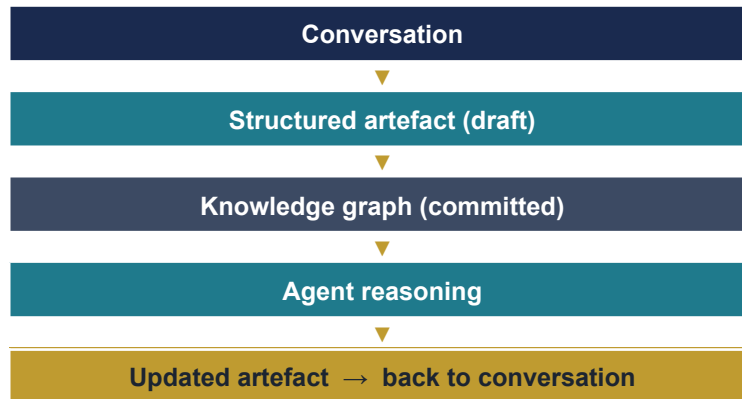
- Workflow contains Tasks
- Task uses Knowledge · Task performed by Agent
- Agent uses Tools · Tool calls API
- Workflow governed by Policies · Policy mitigates Risk
- Decision informed by Evidence · Deliverable derived from Artefacts

Why the graph matters to the Orchestrator

Because artefacts are nodes and their dependencies are edges, the Orchestrator can do two things a flat database cannot: assemble a precise context slice for any agent call by traversing the graph, and propagate change — when an upstream artefact moves, it flags every downstream artefact that now needs review.

7 The conversation–artefact loop

This loop is the heartbeat of the workbench. It replaces the click–fill–save rhythm of a form-based tool with a collaborative cycle that the Orchestrator drives.



The user stories that matter here are **cognitive**, not CRUD: “As an FDE, I want AI to convert interviews into an ontology.” “As an FDE, I want AI to generate candidate workflows.” “As an FDE, I want governance automatically suggested.” Each is a request for the Orchestrator to run a turn of this loop.

A worked turn

- The FDE uploads a stakeholder interview transcript.
- The Orchestrator recognises the engagement is in Discover, routes the transcript to the Discovery Agent with the current project context.
- The Discovery Agent extracts stakeholders, objectives and constraints as a draft artefact and returns clarifying questions.
- The FDE answers and edits the draft on the canvas; the Orchestrator commits it to the knowledge graph with provenance.
- Committing the artefact satisfies the Discover exit gate, so the Orchestrator offers to advance to Purpose — or, if a governance-relevant risk was surfaced, invokes the Governance Agent first.

8 Orchestration patterns

The Orchestrator sequences agents using a small set of named patterns. Which pattern applies is a function of engagement state and user intent.

Pattern	When the Orchestrator uses it
Stage-gated progression	Default forward motion. A stage's exit gate (its committed Outputs) must be satisfied before the next stage opens. The Orchestrator holds the gate.
Fan-out enrichment	Several agents run in parallel on a shared input — e.g. Knowledge and Governance both examine a new document, one extracting ontology, the other flagging data-handling risk.
Generator–critic loop	One agent proposes, another reviews. The Workflow Agent drafts a process; the Governance Agent critiques it against policy; the Orchestrator iterates until the gate passes.
Hierarchical delegation	A complex request is decomposed by the Orchestrator into sub-tasks handed to different agents, then recomposed into a single artefact.
Backward propagation	A late-stage finding invalidates an earlier artefact. The Orchestrator uses the graph to flag every dependent artefact and reopens the affected stages.

9 Governance as a cross-cutting concern

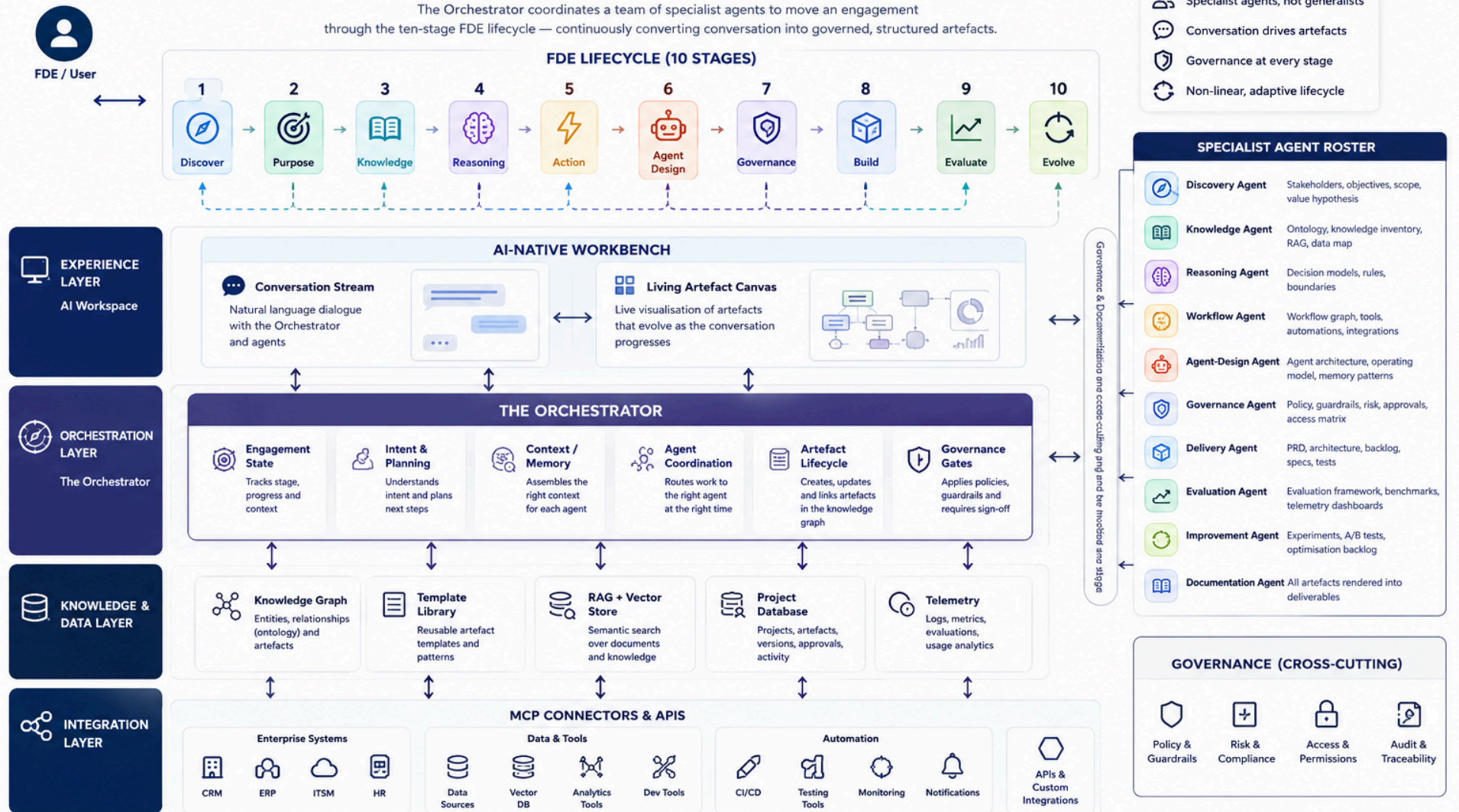
Governance is a lifecycle stage in the methodology, but in the running system it is everywhere. The Orchestrator enforces it continuously rather than at a single checkpoint.

- **Stage gates.** No stage advances until its Outputs are committed and any required approvals are recorded.
- **Guardrails.** The guardrail library constrains what agents may do; the Orchestrator applies the relevant guardrails to every agent call.
- **Access and audit.** The access matrix governs who and which agent may read or write each artefact; every commit is logged with provenance, producing the audit trail by construction.
- **Human-in-the-loop.** Where policy demands it, the Orchestrator pauses and requires explicit sign-off before committing or acting on an artefact.

Open FDE Toolkit

An Orchestrator-Centred Design

The Orchestrator coordinates a team of specialist agents to move an engagement through the ten-stage FDE lifecycle — continuously converting conversation into governed, structured artefacts.



The Orchestrator continuously turns conversation into a governed knowledge graph of interdependent artefacts, coordinating specialist agents across a non-linear ten-stage lifecycle to deliver deployable, governed AI solutions.

10 Build sequence

Consistent with the design-first principle, the build order puts the thinking before the plumbing. Backend and frontend come surprisingly late.

Order	Step	Produces
1	Design the methodology	The ten-stage lifecycle with per-stage questions, outputs and gates
2	Define the artefacts	The reusable artefact per stage — each becomes an object in the system
3	Design the information model	The knowledge-graph schema: entities and relationships (the ontology)
4	Write the user stories	Cognitive stories that map to turns of the conversation–artefact loop
5	Design the AI-first UX	The conversation-plus-canvas workbench, not forms
6	Define the AI agents	The agent roster; the Orchestrator’s routing and coordination logic
7	Build the backend	Projects, workflows, knowledge graph, agent runs, evaluations, telemetry, policies, templates
8	Build the frontend	The dual-pane workspace over the orchestrated backend

In one sentence

The FDE Toolkit is an AI-native workbench whose central Orchestrator turns a conversation into a governed knowledge graph of interdependent artefacts, coordinating specialist agents across a non-linear ten-stage lifecycle to move an ambiguous business problem to a deployable, governed AI solution.